Browser Agent

Coworker Handoff Package for Cursor

Local AI browser-control app using TypeScript, DeepSeek (deepseek-v4-flash), and the official Chrome DevTools MCP server.

Contents of this package

- **browser_agent/** — complete project source (no node_modules, no .env secrets)
- **Browser_Agent_Coworker_Handoff.pdf** — this document (setup + original brief)
- **START_HERE.md** — short checklist to open in Cursor

Do not copy anyone else's DeepSeek API key. Create your own at https://platform.deepseek.com and put it only in a local `.env` file.

1. Restart this project on your Mac with Cursor

Prerequisites

- macOS with Google Chrome (stable)
- Node.js 20+ LTS (`node -v`)
- Cursor IDE
- Your own DeepSeek API key

Steps

```
1. Copy the handoff folder to your machine (or unzip the archive).
2. Open the folder in Cursor:
     File → Open Folder → browser_agent_coworker_handoff/browser_agent
3. In Terminal (inside that folder):

     cp .env.example .env
     # Edit .env and set:
     # DEEPSEEK_API_KEY=sk-your_key_here

     npm install
     npm run dev

4. Open http://127.0.0.1:8821/
5. Setup tab → Connect (launch dedicated Chrome)
6. New Task → use sample URL:
     http://127.0.0.1:8820/samples/example-page.html
7. Live Run → watch steps / approve WRITE if prompted
8. Results → approve record → Export CSV
```

Useful scripts

```
npm run dev        # API :8820 + Vite UI :8821
npm test           # policy / limits / extraction tests
npm run typecheck  # TypeScript
npm run build && npm start   # production UI served from :8820
```

Optional: attach to a dedicated Chrome on port 9222

```
mkdir -p "$HOME/.browser-agent/chrome-profile"
/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome \
  --remote-debugging-port=9222 \
  --user-data-dir="$HOME/.browser-agent/chrome-profile"

# In .env:
CHROME_BROWSER_URL=http://127.0.0.1:9222
```

Never point this app at your everyday Chrome profile. Use `~/.browser-agent/chrome-profile` only.

2. What was built (architecture)

- **Agent Host** — DeepSeek OpenAI-compatible tool-call loop with step/budget/timeout limits
- **Browser Controller** — spawns chrome-devtools-mcp (slim, no usage stats) against a dedicated Chrome profile
- **Extraction Engine** — Zod-validated fields with confidence + evidence; no invented values
- **Policy / Approval** — READ & NAVIGATE auto; WRITE gated; SENSITIVE always requires approval
- **Audit Log** — SQLite (runs, tools, tokens, cost, approvals)
- **UI** — React views: Setup, New Task, Live Run, Results, Audit

- **Export** — CSV and optional CRM push only after record approval

Default ports

- **8820** — Express API (+ sample pages under /samples)
- **8821** — Vite UI (dev)
- **9222** — optional Chrome remote debugging attach

Key paths inside browser_agent/

```
src/server/agent/host.ts          # DeepSeek + MCP tool loop
src/server/policy/                # READ/NAVIGATE/WRITE/SENSITIVE gates
src/server/mcp/                   # chrome-devtools-mcp client
src/shared/schemas.ts             # Zod schemas
src/web/                          # React UI
mcp/chrome-devtools.json          # MCP launch config
samples/example-page.html         # harmless extraction smoke test
SECURITY.md                       # risks and LinkedIn constraints
```

3. Original product brief (for Cursor restart / context)

Paste the following into Cursor if you need the agent to continue development from the original specification:

```
Build a local AI browser-control application using TypeScript, DeepSeek,
and the official Chrome DevTools MCP server.

OBJECTIVE
Create an AI agent that connects to a dedicated Chrome browser, navigates
websites, inspects page content, extracts structured information, and
performs browser actions under explicit user control.

First use case:
1. Open a URL supplied by the user.
2. Inspect the page using Chrome DevTools MCP.
3. Extract user-selected fields into structured JSON.
4. Display extracted information for review.
5. Export approved results to CSV or a configurable CRM API.
6. Never send messages, submit forms, add connections, make purchases,
   delete information, or perform other external side effects without
   explicit user approval.

TECHNOLOGY
- TypeScript, Node.js LTS
- DeepSeek OpenAI-compatible API; model deepseek-v4-flash
- Official package chrome-devtools-mcp
- Official MCP TypeScript SDK
- Zod, SQLite, React (or minimal local web UI)
- Environment variables for secrets

ARCHITECTURE COMPONENTS
1. Agent Host — DeepSeek + MCP tools, max steps/timeout/token budgets
2. Browser Controller — dedicated Chrome profile, headed mode, Stop button
3. Extraction Engine — schema-driven Zod JSON with evidence + confidence
4. Policy and Approval — READ / NAVIGATE / WRITE / SENSITIVE
5. Audit Log — run_id, tools, sanitized args, tokens, cost
6. Cost Controls — default max 20 iterations, $0.02 task budget

CHROME MCP CONFIG
npx -y chrome-devtools-mcp@latest --slim --no-usage-statistics
Optional attach: --browser-url=http://127.0.0.1:9222

DEEPSEEK
DEEPSEEK_API_KEY=
DEEPSEEK_BASE_URL=https://api.deepseek.com
DEEPSEEK_MODEL=deepseek-v4-flash

LINKEDIN SAFETY
Human-in-the-loop research assistant only. Do not bypass controls, scrape
at scale, rotate accounts, solve CAPTCHAs, send automated messages or
connection requests.

UI VIEWS
Setup, New Task, Live Run, Results, Audit

DELIVERABLES
Working source, README (macOS), .env.example, MCP config, DB schema,
Zod schemas, approval/limit tests, sample extraction page, Dockerfile,
SECURITY.md
```

4. Suggested first Cursor prompt on the new machine

```
I opened the browser_agent project from the coworker handoff package.
Please:
1. Confirm prerequisites (Node 20+, Chrome).
```

```
2. Help me create .env from .env.example (I will paste my own DeepSeek key).
3. Run npm install and npm test.
4. Start npm run dev and verify http://127.0.0.1:8821 and :8820.
5. Walk me through the sample extraction against
   http://127.0.0.1:8820/samples/example-page.html

Do not use my normal Chrome profile. Use ~/.browser-agent/chrome-profile.
Do not commit .env. Read SECURITY.md before suggesting LinkedIn workflows.
```

5. Known gotchas learned during MVP bring-up

- **Stale shell API keys:** If `DEEPSEEK_API_KEY` is exported in the shell with an old value, it used to override `.env`. The app now loads `.env` with `override: true`. Still prefer putting the key only in `.env`.
- **401 Authentication Fails:** Invalid DeepSeek key. Create a new key at platform.deepseek.com.
- **Slim MCP tools:** With `--slim`, tools are typically `navigate`, `evaluate`, `screenshot` (not the full DevTools set).
- **tsx watch restarts wipe in-memory runs:** Editing server files mid-run restarts the API and orphans active runs. Reconnect Chrome and start a new task.
- **Ports:** 8820/8821 must be free. Check with `lsof -iTCP:8821 -sTCP:LISTEN`.
- **Sample success fields:** Alex Rivera; Senior Product Manager · B2B SaaS; Northwind Analytics; Portland, OR; https://example.com/people/alex-rivera

6. Security summary

- Dedicated Chrome profile only — never the user's default profile by default
- Page content and tool results are sent to DeepSeek — do not open secret pages you will not share with that provider
- WRITE/SENSITIVE actions require human approval
- No CAPTCHA bypass, fingerprint spoofing, proxy rotation, or LinkedIn automation evasion
- See SECURITY.md in the project for the full note

7. .env.example (copy to .env and fill secrets)

```
DEEPSEEK_API_KEY=
DEEPSEEK_BASE_URL=https://api.deepseek.com
DEEPSEEK_MODEL=deepseek-v4-flash
PORT=8820
HOST=127.0.0.1
DATA_DIR=./data
MAX_ITERATIONS=20
TASK_BUDGET_USD=0.02
TASK_TIMEOUT_MS=300000
WRITE_REQUIRES_APPROVAL=true
CHROME_USER_DATA_DIR=~/.browser-agent/chrome-profile
CHROME_BROWSER_URL=
CRM_ENDPOINT=
CRM_API_KEY=
```

8. Sharing checklist

- Share the entire `browser_agent_coworker_handoff/` folder (or zip it)
- Confirm `.env` is NOT included
- Coworker uses their own DeepSeek billing/key

- Optional: zip with `zip -r browser_agent_coworker_handoff.zip browser_agent_coworker_handoff`